AI/ML Engineer Onboarding, Proof Layer & Agent OS Plan

A practical playbook for joining a multi-company ML platform (lead / churn / NBA and similar), building eval & degradation foundations, and configuring an agent workflow that maximizes leverage without losing judgment.

Compiled 2026-07-27

For personal use in a concrete repo — planning, not product code.

Contents

1. How to use this document
2. Context & strategy (why this plan exists)
3. Career investment principles (quick reference)
4. Repo plan: Eval, health & promotion foundations
5. Personal 30-day onboarding plan (reality-adjusted)
6. Agent OS: skills, commands, loops, harnesses, vault
7. Combined calendar (product ∩ agent setup)
8. Checklists & red flags
9. Ready-to-paste AGENTS.md skeleton

1. How to use this document

This PDF compiles the planning artifacts from recent guidance into one place you can drop into a new repository later (as docs) or keep as a personal runbook.

- **Section 4** → paste into a team planning doc / epic (observability & eval foundation).
- **Section 5** → your personal first-30-days plan when eval dashboards do not exist yet.
- **Section 6** → how to stand up Cursor/agent tooling day-by-day for max efficiency.
- **Section 7** → single calendar merging product work and agent OS setup.

Core metaphor throughout: **you are the engineering manager; agents are the team**. Agents implement and sweep. You own specs, metrics, golden cases, promotion judgment, and merges.

2. Context & strategy

Project reality (inputs)

- Hundreds of models over time: lead, churn, NBA / next-best-action, etc., **per company**.
- Companies roll out slowly; a large share of work is **data readiness / data fixing**.
- **No eval dashboard/harness** yet — no shared offline scoring path.
- **No API/business metrics dashboard** — hard to see live decision quality by company/model.
- **No easy degradation signal** — hard to answer “does it still work?” or “did it get worse?”

Strategic one-liner

> *Fix data to unlock companies, but unlock companies only as fast as we can prove models still work — by building eval, health, and degradation gates before scaling to hundreds of models.*

Your positioning (ceiling work)

In the agent era, implementation is cheap. Value concentrates in **intent** → **specification** → **verification** → **accountability**. On this project, that means becoming the person who creates the minimum proof system: golden eval + baseline + serving sanity + promotion checklist that separates data readiness from model readiness — starting with a ruthless pilot.

3. Career investment principles (quick reference)

Default deliberate-practice split

Bucket	Share	On this project means
Ship one real proof loop	~40%	Eval runner, baselines, checklist, health signals
Evals & proof	~25%	Golden set from real company failures; worst-slice
Deep ML / domain / data	~20%	Why companies fail promotion; online/offline gaps
Agentic craft	~10%	Spec → agent implements → you verify
Reading / courses	~5%	Only if it unblocks this week's artifact

Rule: No study hour without a linked artifact.

Floor vs ceiling

Getting cheap (floor)	Staying scarce (ceiling)
Boilerplate, scaffolding, standard train-and-ship	Eval / proof design; promotion gates
Framework tourism; polyglot syntax	Objective-bound judgment; worst-slice diagnosis
Dashboards without cases underneath	Accountable release ownership; data vs model readiness
Prompt-only "AI fluency"	Domain + production reality; agent orchestration with proof

Trust model for agents (always)

- **Fresh-context review** — never trust the authoring session to certify its own work.
- **Escalate** ambiguous or product-changing decisions to you.
- **End-to-end evidence** — CLI output, report.json, health query — not only unit tests.
- **Auditable** — one task → one branch → one PR; evidence attached.

4. Repo plan: Eval, health & promotion foundations

Paste-ready planning text for a concrete repository (e.g. `docs/plans/observability-and-eval-foundation.md` or a Linear epic).

4.1 Problem statement

We can train and serve many models, but we cannot systematically verify, compare, or monitor them. Promotion of new companies is gated on data fixes, while model health after promotion is largely invisible. That makes every rollout a bet instead of a controlled change.

4.2 Goals (1–2 quarters)

Must-have outcomes

- **Per-model offline eval harness** — versioned datasets + metrics + CLI/CI → machine-readable report (pass/fail + slices).
- **Minimal serving health signals** — traffic, errors, latency, prediction sanity per company and model type.
- **Degradation detection v0** — current vs pinned baseline; alert/block on primary or worst-slice regression.
- **Promotion checklist** — explicit gates for company data readiness and model readiness (manual OK at first).

Non-goals for v0

- Polished BI for every business KPI
- Fully automated multi-agent training orchestration
- Perfect causal attribution of every outcome
- One mega-platform for all model types on day one

Start with **one model family × a few companies**, then generalize.

4.3 Design principles

- 1. **Proof before scale** — do not accelerate company rollout without a way to detect bad models.
- 2. **Worst-slice over averages** — report overall and per-company / per-segment metrics.
- 3. **Baseline + delta** — every eval run compares to a pinned baseline artifact.
- 4. **Separate data-readiness from model-readiness** — both must pass to promote a company.
- 5. **Artifacts over opinions** — promotion leaves a report (eval JSON, metric snapshot, checklist).
- 6. **Human gate on ambiguous product changes** — automation blocks clear regressions; humans decide tradeoffs.

4.4 Workstreams

W1 — Offline eval harness (highest leverage)

- evals/ with versioned golden sets, metric defs, runner: evaluate --company X --model-type Y --baseline Z → report.json
- CI or scheduled job fails on clear regressions
- Doc: how to add a case; how to interpret a report
- Slices: by company; historically breaking segments; abstain/low-confidence if applicable
- **DoD:** one command → pass/fail + worst slice; baseline stored; fault injection catches a bad model

W2 — Serving & API health (MVP observability)

- Metrics: rate, 4xx/5xx, latency p50/p95; counts per company_id + model_type; null/constant prediction rates
- One operational view (Grafana, SQL job, or daily notebook — fine for v0)
- Alerts: error spike, zero traffic for active company, sanity breach
- **DoD:** answer in <5 min whether company C / model M has healthy traffic and non-degenerate outputs

W3 — Business / decision metrics (thin wedge)

- Define 1–3 north-star proxies per model family (not a full BI suite)
- Log decisions with join keys; weekly report per company (CSV/Slack OK)
- **DoD:** for pilot companies, notice a sustained drop in a weekly proxy

W4 — Degradation & promotion gates

- PromotionChecklist: Data readiness (schema, nulls, labels, leakage, volume, known bad fields) + Model readiness (eval vs baseline, slice floors, soak) + owner sign-off
- Hard block on offline regression beyond threshold; warn on sanity/proxy drift
- Registry sketch: live model version per company (YAML/table OK)

W5 — Data-fix throughput (parallel, not a substitute)

- Couple data-fix epics to gates: each fix defines which eval cases / readiness checks unlock
- “Data fixed” is not done until company passes data-readiness checks used by promotion

4.5 Phased roadmap

Phase	Focus	Approx.
0 Inventory	Catalog families/companies; folklore of failure detection; pick pilot (1 family × 3–5 companies)	1–2 wks
1 Eval harness v0	Golden set + metrics + CLI + baseline + fault-injection demo	2–4 wks
2 Serving health v0	Metrics + one board/query + 2–3 alerts (parallel with Phase 1 if possible)	parallel
3 Degradation + checklist	Wire eval into promotion; soak; one real/dry-run promotion	2–3 wks
4 Business proxy wedge	Join keys + weekly proxy (or explicit NO LABELS gap doc)	2–4 wks
5 Generalize	Second family template; company onboarding runbook	ongoing

4.6 Pilot success criteria (after Phase 3)

- Evaluate any pilot company offline in one command
- Detect a synthetic regression (fault injection) automatically
- See whether each pilot company gets healthy serving traffic
- Company promotion ticket includes data + model readiness artifacts
- No longer rely solely on “someone notices” for model breakage

4.7 Suggested immediate tickets

- Inventory: model types × companies × current detect-failure methods
- Choose pilot model family + companies; write success metrics

- Define v0 metric spec (primary + slices + thresholds as hypotheses)
- Scaffold eval runner + report schema
- Collect first 20–50 golden cases (including known failures)
- Emit/confirm serving metrics for pilot path
- Draft promotion checklist (data + model) as a living doc
- Fault-inject a bad model and show the gate catch it

5. Personal 30-day onboarding plan (reality-adjusted)

Same structure as a generic NBA-like onboarding plan, but **building the missing proof layer is the job** — not finding an existing dashboard.

North star (day 30)

- You can explain the decision/serving loop for the pilot model family.
- You can describe how we *will* know a model is bad (even if v0 is CLI + spreadsheet).
- You have a golden set and a baseline for pilot companies.
- You have a draft promotion checklist separating data vs model readiness.
- You own or co-own eval/degradation for the pilot — not merely "I read the codebase."

Days 0–2 — Boot + map

- Get repo running; run existing tests/CI.
- Map training entrypoints, serving/API, config per company, data validation (if any).
- Inventory folklore: how do people notice breakage today?
- **Artifact:** System card v0 + rough inventory table (model_family × company × status).

Days 3–5 — Trace + pain list

- Trace one live/staging request for one company end-to-end.
- Trace training → artifact → serve path.
- Write pain list: cannot tell if… / wouldn't notice if…
- Propose pilot: 1 model family + 3–5 companies (include one painful data company).
- **Do not** start fine-tuning or multi-company refactors yet.
- Agents may find call sites; **you** write invariants in your own words.

Days 6–12 — Eval harness v0 (main investment)

- Metric spec (1 page): primary, 2–3 slices, fail hypotheses.
- Golden set 20–50 cases from real failures/edges.
- Runner: one command → report.json (overall + slices + vs baseline).
- Baseline pin + fault injection demo (show red report).
- Agents scaffold; you define metrics, cases, fail rules.

Days 8–14 (parallel) — Serving health wedge

- Confirm/add logs: company_id, model_type, latency, status, prediction summary.
- Any view: Grafana / SQL / daily script / notebook.
- 2–3 alerts max: error spike, zero traffic, degenerate outputs.
- **Artifact:** 5-minute health check runbook for pilot companies.

Days 13–18 — Promotion checklist

- Data readiness draft + model readiness draft.
- Align with data-fix owners: each fix unlocks specific checks/cases.

• Dry-run one company through the checklist.

Days 19–24 — First degradation path

• Provisional regression policy; wire eval into CI or schedule.
• Thin business proxy if labels exist; else document the gap.
• Write: “How we know model M for company C is healthy.”

Days 25–30 — Claim ownership + 60-day plan

• Propose owning pilot eval + degradation; co-own checklist with data.
• Pick one 60-day focus: second family template OR deepen proxies OR scale readiness checks.
• Portfolio artifact: before/after — we couldn’t detect degradation; now we can for pilot.

Weekly time split while onboarding

Block	Time	What
Trace / ship proof	2.5–3.5h	Harness, checklist, health, one proof-backed ticket
Evals & cases	1.5–2h	Grow golden set; slices
Domain / data reality	1–1.5h	Promotion blockers; objective gaps
Agentic craft	30–45m	Spec writing; diff review drill
Read	≤30m	Only docs that unblock today’s artifact

6. Agent OS: skills, commands, loops, harnesses, vault

Map agent tooling onto the same days so you maximize efficiency without building a tooling cathedral before you have a pilot and a proof loop.

6.1 Layer map

Layer	Purpose	Examples
AGENTS.md / rules	Always-on repo law	Pilot scope; don't invent metrics; escalate promotion
Knowledge vault	Durable facts agents must read	System card, inventory, metric specs, failure folklore
Skills	How to do recurring kinds of work	add-eval-case, promotion-check, trace-decision
Commands	One-shot slash/CLI entrypoints	/map-loop, /eval-run, /health-check
Loops	Multi-step orchestration	Implement→validate; overnight ablation
Harnesses	Machine gates agents must pass	make check, evals/run, CI, capability verify
Hooks	Automatic guardrails	Remind evidence; block dangerous actions

Build order: vault + AGENTS.md → harness stubs → commands → skills → loops → hooks. Don't write 10 skills on day 1.

6.2 Target layout (create gradually)

```
AGENTS.md
docs/plans/observability-and-eval-foundation.md
docs/onboarding/system-card.md
docs/onboarding/inventory.md
docs/onboarding/pain-list.md
knowledge/INDEX.md
knowledge/platform-core/
knowledge/data-readiness/
knowledge/model-families/
knowledge/evals/
.cursor/rules/  .cursor/commands/  .cursor/skills/  .cursor/hooks.json
evals/datasets/  evals/baselines/  evals/reports/
scripts/agent/
```

6.3 Days 0–2 — Bootstrap (≤90 min tooling)

- Create short AGENTS.md + system-card template + knowledge/INDEX stubs.
- One rule: read system-card + INDEX before coding.
- Commands only: /map-repo, /run-smoke.
- No overnight loops. Document existing harness command.
- ≤15% time on agent config; ≥85% on boot + map.

6.4 Days 3–5 — Trace + inventory

- Vault: system-card, inventory, pain-list, platform-core, data-readiness notes.
- Commands: /trace-decision, /inventory-pass.
- Skill (1): trace-decision with required output format.
- Optional explore subagent for enums while you trace manually.

6.5 Days 6–12 — Eval harness is the product

- Code harness under evals/ (spec, datasets, baselines, run → report.json, fault_inject).
- Commands: /eval-spec, /eval-add-case, /eval-run, /fault-inject.
- Skills: design-eval-metric, add-golden-case, interpret-eval-report.
- Loops: implement (cloud/gnhf-style) + validate (fresh review + /eval-run evidence).

Efficiency tip: one command that produces report.json beats five skills about evaluation theory.

6.6 Days 8–14 — Health wedge in Agent OS

- scripts/health_check → artifacts/health/<date>.json
- Commands: /health-check; skill: serving-sanity
- Optional hook on stop: if api/serving changed, remind to attach health output

6.7 Days 13–18 — Promotion checklist as agent-readable law

- Vault rules for data-readiness and model-readiness.
- Commands: /promotion-check, /data-fix-to-cases.
- Skills: promotion-gate, company-onboarding.
- AGENTS.md: Data fixed ≠ done until readiness checks + eval baseline exist.

6.8 Days 19–24 — Degradation loops

- CI/schedule eval vs baseline; thresholds.yaml provisional.
- Commands: /degradation-report, /ablation.
- Overnight OK only for bounded tasks: golden-case expansion OR ablation (stops for your judgment).
- Parallelize case collection & CI wiring — never parallelize “invent the metric.”

6.9 Days 25–30 — Harden & freeze

- Finalize DoD for agent PRs; complete INDEX; teammate-facing agent-os.md.
- Add hooks only if pain is real. Avoid skill sprawl and dashboard-first skills.

6.10 Canonical workflows

A. Any feature PR

```
spec → agent implements → fresh review → tests + /eval-run and/or /health-check → you merge
```

B. Is company C OK?

```
/promotion-check --company C → fill gaps as tickets → never promote on vibes
```

C. Did the model get worse?

```
/eval-run --baseline pinned → /health-check → /degradation-report → you interpret worst-slice
```

D. Overnight (after ~day 12)

```
Cloud: add golden cases from incidents; do not change metrics
Morning: you accept/reject → optional ablation agent runs; you decide
```

6.11 Efficiency rules

Do	Don’t
Grow vault from real traces/failures	Write 15 skills on day 1
One harness command > pretty dashboard	Dashboard-first
You spec metrics; agents implement runners	Let agents choose success metrics
Parallelize cases & CI wiring	Parallelize strategy
Overnight only if bounded & reversible	Overnight “refactor serving”

7. Combined calendar (product ∩ agent setup)

Days	Product focus	Agent OS focus
0–2	Boot, map	AGENTS.md, system-card, /map-repo, /run-smoke
3–5	Trace, inventory, pilot pick	Vault cores, /trace-decision, /inventory-pass
6–12	Eval harness v0 + golden set	/eval-*, eval skills, implement+validate loops
8–14	Serving health wedge	/health-check, sanity skill, light hooks
13–18	Promotion checklist	/promotion-check, data-readiness rules
19–24	Degradation path	/degradation-report, /ablation, CI eval
25–30	Own pilot + 60d plan	Freeze commands, agent-os.md, trim cruft

Agent OS “done” by day 30

- AGENTS.md + knowledge INDEX that keep agents on rails
- /eval-run producing baseline deltas for a pilot
- /health-check answering “is C alive and non-degenerate?”
- /promotion-check separating data vs model readiness
- Validate loop (fresh review + evidence) on every meaningful PR
- At most one overnight loop type that stops for your judgment

You do **not** need: full skill marketplace, multi-agent mesh, or a metrics palace.

8. Checklists & red flags

Minimal personal checklist

Week	Must produce
1	Boot → loop diagram → one event/request trace → invariants + inventory folklore
2	Golden set v0 → metric spec → promotion criteria draft → objective/pain notes
3	First proof-backed PR (harness/gate/health) + fault-injection demo
4	Claim owned pilot loop + 60-day plan + agent-os runbook for teammates

Monthly skill self-check

- [ ] I can write a machine-checkable acceptance spec in <30 minutes
- [ ] I maintain a versioned eval set tied to real failures
- [ ] I diagnose failures by layer (data / model / serve / label), not vibes
- [ ] I use agents for sweeps/scaffolding without losing understanding of what shipped
- [ ] I have one proprietary/domain edge I can explain without jargon
- [ ] I have a written artifact (doc, harness, postmortem) from the last 30 days

If fewer than four are true, cut courses and build the missing artifact.

Red flags — investing wrong

- Week 2 and you're fine-tuning architectures with no golden set
- You can "use the agent" but can't explain last week's merged diff
- All metrics are means; no worst-slice
- Building dashboards with no eval cases underneath
- "We'll monitor later" while onboarding more companies
- Data marked fixed with no readiness check
- Supporting all model types at once before a pilot gate works
- Writing 15 agent skills before one harness command exists

Questions to ask the team (day 1–3)

- 1. What is the production decision unit and what gets logged?
- 2. What is forbidden in learning/serving code (leakage, features, labels)?
- 3. What gate (if any) promotes a policy/model/company today?
- 4. What was the last bad ship and how did we catch it (or not)?
- 5. Where do offline metrics lie?
- 6. Who owns on-call / drift / retrain / data promotion?
- 7. What is the feature/schema contract per company?
- 8. How do we know a model is healthy for company C right now?
- 9. What should a new person not touch for two weeks?
- 10. What proof do you want on every AI-assisted PR?

9. Ready-to-paste AGENTS.md skeleton

```
# Agent instructions

## Product
Multi-tenant models (lead/churn/NBA/…). Companies roll out as data becomes ready.

## Hard constraints
- Pilot scope only unless human expands it
- Never invent business KPIs; use approved metric spec
- Data readiness ≠ model readiness; both required to promote
- Always report worst-slice, not mean only
- Attach evidence (eval report and/or health output) for model/serve PRs
- Escalate ambiguous product/threshold/promotion decisions to the human

## Before coding
Read: docs/onboarding/system-card.md, knowledge/INDEX.md,
      relevant domain rules (evals, data-readiness, platform-core)

## Commands
- /map-repo /trace-decision /eval-run /health-check /promotion-check
- /eval-add-case /fault-inject /degradation-report /ablation

## Trust model
- Fresh-context review for every meaningful PR
- End-to-end evidence required (not only unit tests)
- One task → one branch → one PR; keep work auditable

## Definition of done
Tests green + evidence artifact + fresh review
+ human approval on product/threshold/promotion changes
```

Skill template (keep short & local)

```
# Skill: add-golden-case
When: production bug, bad rollout, silent wrong score
Inputs: company_id, model_type, example payload/log, expected behavior
Steps:
1. Read knowledge/evals/rules.md
2. Do not change primary metric without human approval
3. Add case under evals/datasets/pilot/
4. Run eval for that case; attach snippet
Output: case path + before/after report fragment
Escalate: if expected label is disputed
```

Bottom line

On this project, wise investment is not “learn the missing dashboards.” It is **become the person who creates the minimum proof system**: golden eval + baseline + serving sanity + promotion checklist (data vs model) — ruthlessly piloted, then templated to hundreds of models/companies. Configure agents to accelerate scaffolding and sweeps; never let them own success metrics, promotion judgment, or your understanding of what shipped.

— End of plan —